

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе
«Решение СЛУ методом Гаусса»

Выполнил:

студент группы 3825Б1ПМиИ1
Первушин И.Д.

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение.....	3
Постановка задачи.....	4
Руководство пользователя	5
Описание программной реализации	6
Результаты экспериментов	9
Заключение.....	12
Литература	13
Приложение	14

Введение

Системы линейных уравнений чрезвычайно важны и встречаются как в линейной алгебре, так и в матанализе и дискретной математике. Поэтому возникает необходимость эффективно находить их решения. Один из самых простых и достаточно эффективных алгоритмов для решения СЛУ – это метод Гаусса. Его сложность – $O(n^3)$ от размера матрицы. Этот метод ещё называют методом исключения неизвестных ведь в ходе выполнения мы сначала исключаем первую неизвестную из всех строк кроме первой при помощи эквивалентных преобразований, затем вторую из всех строк кроме второй и так далее. В конце получится диагональная матрица и запись в i строке будет $a_{ii} \cdot x_i = c_i$, поделив на a_{ii} можно вычислить все x_i .

Замечание: есть более эффективный способ нахождения вектора ответа – это умножение обратной к A матрицы на правую часть. Если предположить матрицу A^{-1} , то можно будет выполнять метод solve за квадрат операций.

Постановка задачи

1. Создать шаблонный класс `vector`
2. Реализовать класс матрицы, который является вектором векторов
3. Реализовать класс СЛАУ, который является наследником матрицы, но содержит метод гаусса, который принимает правую часть как аргумент и возвращает вектор ответа

Руководство пользователя

Чтобы начать работу, нужно запустить программу, ввести размер матрицы, элементы матрицы а затем вектор b. Программа выведет вектор ответа и попросит ввести следующий вектор b. В случае, если система несовместна (определитель равен 0), программа сообщит об этом и завершит работу.

```
enter size: 5
enter elements:
5 4 3 2 1
4 5 4 3 2
3 4 5 4 3
2 3 4 5 4
1 2 3 4 5
enter b:
15 18 19 18 15
ans vector is
1 1 1 1 1
enter b:
```

1. ввод матрицы 5 на 5, ответ: единичный вектор

Описание программной реализации

Программа содержит 4 файла: gauss2, vector.h, Matrix.h, SLAE.h

Vector.h

Этот файл содержит реализацию шаблонного класса vector. Он имеет поле указателя на данные и размер. У вектора есть 5 конструкторов

- Конструктор по умолчанию
- Конструктор через `std::initializer_list`
- Конструктор с размером и значением. Значение присваивается каждому элементу
- Конструктор с размером
- Конструктор копирования

Особенным может быть конструктор через `initializer_list`. Он нужен чтобы элементы вектора можно было вводить прямо в программе через фигурные скобки, как у вектора из `std`.



2. инициализация вектора через `initializer_list`

`Initializer_list<T>` – контейнер по которому можно только итерироваться при помощи оператора `:` и узнать его размер. Реализация этого конструктора:

```
Vector(std::initializer_list<T> t) {  
    this->n = t.size();  
    a = new T[n];  
    int ii = 0;  
    for (T i : t) {  
        a[ii] = i;  
        ii++;  
    }  
}
```

Также у вектора перегружены операторы:

- Оператор присваивания
- Доступ по индексу в квадратных скобках с проверкой корректности индекса
- Умножение на скаляр `*=`, `*`
- Вычитание вектора `-=`
- Операторы ввода и вывода

Оператор ввода реализован так:

```
template<typename T>
```

```
std::istream& operator>>(std::istream& is, Vector<T>& v) {
    for (int i = 0; i < v.size(); ++i)
        is >> v[i];
    return is;
}
```

Так как у вектора перегружены квадратные скобки и есть геттер на размер, этот метод не обязательно было делать friendly к вектору.

Кроме этого у него есть методы:

- `void swap(Vector& other)` – взаимно меняет размеры векторов и их ссылки на память, работает за $O(1)$.
- `void print()` – работает как `operator>>`, но выводит каждый элемент на новой строке, что позволяет выводить `vector<vector<T>>` как матрицу, вызвав этот метод. Он циклом пройдёт по элементам, каждый из которых `vector<T>` и тот корректно выведется благодаря перегрузке оператора вывода.
- `int size()` – геттер для поля `n`.

Метод swap реализован следующим образом:

```
void swap(Vector& other) {
    T* tmp_a = a;
    a = other.a;
    other.a = tmp_a;
    int tmp_n = n;
    n = other.n;
    other.n = tmp_n;
}
```

Matrix.h

Этот файл содержит реализацию шаблонного класса квадратная матрица размера `n`. Она содержит размер и матрицу `a`, которая выглядит как `Vector<Vector<T>>` `a`.

У матрицы есть конструкторы:

- конструктор с размером и значением, которое заполняет матрицу
- Конструктор по размеру
- Конструктор с `initializer_list`
- Конструктор копирования
- Конструктор от вектора векторов

Большинство конструкторов написаны очень просто благодаря классу вектор. Например, по размеру достаточно написать `a = Vector<Vector<T>>(n, Vector<T>(n));` Это вызовет конструктор у вектора по размеру и значению, а каждым значением будет вектор, созданный только по размеру.

Конструктора копирования, оператора присваивания и деструктора у класса матрицы нет, так как он не взаимодействует напрямую с памятью и хранит не указатели, а векторы, для которых все эти операции определены правильно. Метод ввода с консоли и вывода на консоль также ссылаются на эти методы вектора.

SLAE.h

Этот файл содержит реализацию шаблонного класса система линейных алгебраических уравнений, наследуемый от матрицы. Он содержит конструктор по размеру и от матрицы, которые реализованы через конструкторы класса матрица и единственный метод `Vector<T> solve(const Vector<T> &num)`. Первым делом метод проверяет совпадение размеров вектора num с размером матрицы класса и в случае различия выкидывает исключение с соответствующим текстом ошибки. (Примечание: из-за особенностей оператора `>>` это исключение не выкинется, потому что оператор считывает ровно столько сколько нужно. Это исключение может появиться если вычислять прямо в программе). Затем, чтобы не изменить матрицу a и переданный по ссылке num, создаётся отдельная матрица b и вектор c при помощи конструкторов копирования. Затем алгоритм проходится вперёд и вниз, выбирая ненулевой ведущий элемент. Если элемент нулевой, он идёт вниз и находит первый ненулевой. Из равенства всех элементов нулю следует, что определитель матрицы равен 0 и по теореме Крамера система либо несовместна, либо имеет бесконечно много решений. Затем выбранная строка делится на ведущий элемент и вычитается из всех нижних так, чтобы ниже ведущего элемента оказались нули. Потом переходит к следующему столбцу. Конечно, все эти вычисления параллельно выполняются и с вектором правой части. После приведения матрицы к треугольной и единицами по диагонали, выполняется проход назад и вверх, зануляя все элементы над ведущим. В итоге матрица b становится единичной, а правая часть становится вектором ответа, который и нужно вернуть.

```
enter size: 5
enter elements:
5 4 3 2 1      1 0.8 0.6 0.4 0.2
4 5 4 3 2      0 1 0.888889 0.777778 0.666667
3 4 5 4 3      0 0 1 0.875 0.75
2 3 4 5 4      0 0 0 1 0.857143
1 2 3 4 5      0 0 0 2.22045e-16 1.71429
enter b:
15 18 19 18 15
1 0.8 0.6 0.4 0.2
0 1.8 1.6 1.4 1.2
0 1.6 3.2 2.8 2.4
0 1.4 2.8 4.2 3.6
0 1.2 2.4 3.6 4.8

1 0.8 0.6 0.4 0.2
0 1 0.888889 0.777778 0.666667
0 0 1.77778 1.55556 1.33333
0 0 1.55556 3.11111 2.66667
0 0 1.33333 2.66667 4

1 0.8 0.6 0.4 0.2
0 1 0.888889 0.777778 0.666667
0 0 1 0.875 0.75
0 0 0 1.75 1.5
0 0 0 1.5 3

ans vector is
1 1 1 1 1
```

3. процесс приведения матрицы к треугольному виду

Можно заметить, что при вычислениях возникает погрешность в 10^{-16} , однако на ответ это не влияет. Также может показаться, что точнее будет сначала вычитать и делить в самом конце, однако для матриц с маленькими дробными числами может быть эффективнее наоборот сначала делить и только потом вычитать, так что разницы в последовательности операций нет.

gauss2.cpp

Этот файл содержит основную программу. Благодаря большому количеству перегруженных методов у всех классов, код очень простой и понятный. Сначала считывается размер матрицы с проверкой чтобы он был положительным. Затем создаётся вектор и матрица через конструкторы по размеру. СЛАУ считывает значения с консоли. Затем в бесконечном цикле в try считывается правая часть и выводится результат работы метода solve от SLAE. При какой-либо ошибке она отлавливается и выводится её сообщение. Так как ошибкой может быть равенство определителя матрицы нулю и матрицу придётся вводить заново, программа завершается.

Результаты экспериментов

Для проверки корректности программы я провёл несколько тестов с различными матрицами.

Вырожденные матрицы

Я вводил в программу следующие вырожденные матрицы:

1	2	3	1	2	3	1	2
1	2	3	0	0	0	2	4
4	5	6	4	5	6		

Для всех них программа определила вырожденность и написала сообщение об ошибке.

```
enter size: 4
enter elements:
1 0 1 0
0 1 0 1
1 1 1 1
0 1 0 1
enter b:
2 3 5 3
Err: det(A) = 0
```

4. Вывод сообщения об ошибке для вырожденной матрицы

Обыкновенные матрицы

В программе было протестировано много целочисленных матриц с целыми решениями, в том числе и с нулями на месте ведущих элементов. Размер матрицы был от 1 до 5.

<pre>enter size: 3 enter elements: 5 -3 2 1 4 -1 2 -1 4 enter b: 17 -10 16 ans vector is 1 -2 3</pre>	<pre>enter size: 3 enter elements: 0 1 1 1 0 1 1 1 0 enter b: 2 2 2 ans vector is 1 1 1</pre>	<pre>enter size: 5 enter elements: 5 4 3 2 1 4 5 4 3 2 3 4 5 4 3 2 3 4 5 4 1 2 3 4 5 enter b: 15 18 19 18 15 ans vector is 1 1 1 1 1</pre>
<pre>enter size: 2 enter elements: 2 1 1 -1 enter b: 1 -4 ans vector is -1 3</pre>	<pre>enter size: 1 enter elements: 5 enter b: 10 ans vector is 2</pre>	<pre>enter size: 4 enter elements: 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 enter b: 9 8 7 6 ans vector is 1 2 3 4</pre>

5. Решение целочисленных матриц

Для всех этих матриц программа вывела правильный ответ

Дробные матрицы

Также программа была проверена на дробных матрицах, которые были почти вырождены и плохо обусловлены.

<pre>enter size: 4 enter elements: 1.0 0.5 0.333333333333 0.25 0.5 0.333333333333 0.25 0.2 0.333333333333 0.25 0.2 0.166666666667 0.25 0.2 0.166666666667 0.142857142857 enter b: 2.08333333333 1.28333333333 0.95 0.759523809524 ans vector is 1 1 1 1</pre>	<pre>enter size: 4 enter elements: 1 2 3 4 0.001 1 2 3 0 0.001 1 2 0 0 0.001 1 enter b: 10 6.001 3.001 1.001 ans vector is 1 1 1 1</pre>	
<pre>enter size: 3 enter elements: 0.0001 0.0002 0.0003 0.0002 0.0003 0.0001 0.0003 0.0001 0.0002 enter b: 0.0014 0.0011 0.0011 ans vector is 1 2 3</pre>	<pre>enter size: 3 enter elements: 1e-6 2e-6 3e-6 2e-6 3e-6 1e-6 3e-6 1e-6 2e-6 enter b: 1.4e-5 1.1e-5 1.1e-5 ans vector is 1 2 3</pre>	<pre>enter size: 2 enter elements: 1e-8 1 1 1 enter b: 1.00000001 2 ans vector is 1 1</pre>

Матрицы с очень маленькими коэффициентами

Можно заметить, что программа работает корректно даже с коэффициентами порядка 10^{-8} . При более маленьких коэффициентах появляются ошибки, например при 10^{-12} .

```
enter size: 3
enter elements:
1e-12 1 0
0 1e-12 1
1 0 1e-12
enter b:
1.00000000000001 1.00000000000001 1.00000000000001
ans vector is
1.2207e+08 0.999878 1
```

Правильный ответ: единичный вектор

Выводы из экспериментов

Программа корректно работает для коэффициентов до 10^{-12} , что удовлетворяет большинству задач. Также программу можно использовать для проверки вырожденности матрицы a .

Заключение

В итоге получилась работающая библиотека для решения СЛАУ методом Гаусса, завёрнутая в удобные шаблонные классы вектора и матрицы. Основной плюс в том, что весь код написан с нуля — от динамического массива до операторов ввода-вывода, и он действительно решает системы, причём не только с целыми числами, но и с дробными, вплоть до коэффициентов порядка 10^{-10} , что для `double` довольно неплохо. Программа ловит вырожденные матрицы и кидает исключение, а благодаря перестановке строк при нуле на диагонали не падает на системах, где ведущий элемент оказался нулевым. Правда, настоящего выбора главного элемента по модулю я не делал, только искал первый ненулевой вниз, но для большинства учебных примеров этого хватило, и даже почти вырожденные матрицы решались с приемлемой точностью. Эксперименты показали, что при коэффициентах меньше 10^{-12} погрешность начинает заметно расти — это уже проблема самого типа `double`, а не алгоритма. В целом метод Гаусса в моей реализации полностью работоспособен, а шаблонность позволяет при необходимости подставить вместо `double` что угодно с арифметическими операциями, хоть комплексные числа или даже векторы.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.
3. Грэхель, Кнут. Конкретная математика

Приложение

Матрицы для тестирования программы:

3

1 0.0001 0.0002

0.0001 1 0.0003

0.0002 0.0003 1

1.0003 1.0004 1.0005

С ответом 1 1 1

3

1e-8 1 0

0 1e-8 1

1 0 1e-8

1.000000001 1.000000001 1.000000001

С ответом 1 1 1

3

0.5 0.3333333333 0.25

0.3333333333 0.25 0.2

0.25 0.2 0.1666666667

1.0833333333 0.7833333333 0.6166666667

С ответом 1 1 1

4

1 0 1 0

0 1 0 1

1 1 1 1

0 1 0 1

2 3 5 3

Вырожденная

(достаточно скопировать строчки и вставить в консоль)

```
#pragma once
```

```
#include <iostream>
```

```
#include <initializer_list>
```

```
template<typename T>
```

```

class Vector {
private:
    T* a;
    int n;
public:
    // конструктор по умолчанию
    Vector() : a(nullptr), n(0) {}
    // конструктор с initializer_list
    Vector(std::initializer_list<T> t) {
        this->n = t.size();
        a = new T[n];
        int ii = 0;
        for (T i : t) {
            a[ii] = i;
            ii++;
        }
    }
    // конструктор с размером и значением
    Vector(int n, T k) : a(new T[n]), n(n) {
        for (int i = 0; i < n; ++i)
            a[i] = k;
    }
    //конструктор по размеру
    Vector(int n) : a(new T[n]), n(n) {}
    // конструктор копирования
    Vector(const Vector& other) : a(new T[other.n]), n(other.n) {
        for (int i = 0; i < n; ++i)
            a[i] = other.a[i];
    }

    // оператор присваивания
    Vector& operator=(const Vector& other) {
        if (this != &other) {
            delete[] a;
            n = other.n;
            a = new T[n];
            for (int i = 0; i < n; ++i)
                a[i] = other.a[i];
        }
    }
}

```

```

        return *this;
    }

    // деструктор
    ~Vector() {
        delete[] a;
    }

    // доступ по индексу
    T& operator[](int i) {
        if (i < 0 || i >= n) {
            throw "wrong index";
        }
        return a[i];
    }
    const T& operator[](int i) const {
        if (i < 0 || i >= n) {
            throw "wrong index";
        }
        return a[i];
    }

    // умножение на скаляр
    void operator*=(T c) {
        for (int i = 0; i < n; ++i)
            a[i] *= c;
    }

    // вычитание вектора
    void operator--=(const Vector& other) {
        for (int i = 0; i < n; ++i)
            a[i] -= other.a[i];
    }

    // умножение на скаляр
    Vector operator*(T c) const {
        Vector res (*this);
        res *= c;
        return res;
    }

```



```

// swap векторов
void swap(Vector& other) {
    T* tmp_a = a;
    a = other.a;
    other.a = tmp_a;
    int tmp_n = n;
    n = other.n;
    other.n = tmp_n;
}

// печать
void print() const {
    for (int i = 0; i < n; ++i)
        std::cout << a[i] << "\n";
    std::cout << "\n";
}

int size() const { return n; }
};

template<typename T>
std::ostream& operator<<(std::ostream& os, const Vector<T>& v) {
    for (int i = 0; i < v.size(); ++i)
        os << v[i] << " ";
    return os;
}

template<typename T>
std::istream& operator>>(std::istream& is, Vector<T>& v) {
    for (int i = 0; i < v.size(); ++i)
        is >> v[i];
    return is;
}

Файл SLAE

//метод Гаусса
Vector<T> solve(const Vector<T> &num) {
    if (num.size() != this->size()) {
        throw "Err: scale of num don't mathes with scale of matrix";
    }
    Vector<T> c(num);
    int n = this->size();
    Matrix<T> b(this->a);

```

```

//go forward, step type
for (int i = 0; i < n; i++) {
    T t = b[i][i];
    if (t == 0) {
        bool fl = 1;
        for (int j = i + 1; j < n; ++j) {
            if (b[j][i] != 0) {
                b[i].swap(b[j]);
                T tmp = c[i];
                c[i] = c[j];
                c[j] = tmp;
                fl = 0;
                break;
            }
        }
        if (fl) {
            throw "Err: det(A) = 0";
        }
    }
    t = b[i][i];
    b[i] *= (1.0 / t);
    c[i] /= t;

    //go down
    for (int j = i + 1; j < n; j++) {
        c[j] -= c[i] * b[j][i];
        b[j] -= (b[i] * b[j][i]);
    }
    //b.print();
}

//go back
for (int i = n - 1; i > 0; --i) {
    //go up
    for (int j = i - 1; j >= 0; --j) {
        c[j] -= c[i] * b[j][i];
        b[j] -= b[i] * b[j][i];
    }
}

return c;

```

}